Name: M. Sivateja
Hall.No:2303A52145
Batch: 41

Task 1 – Rapid App Prototyping with AI

PROMPT

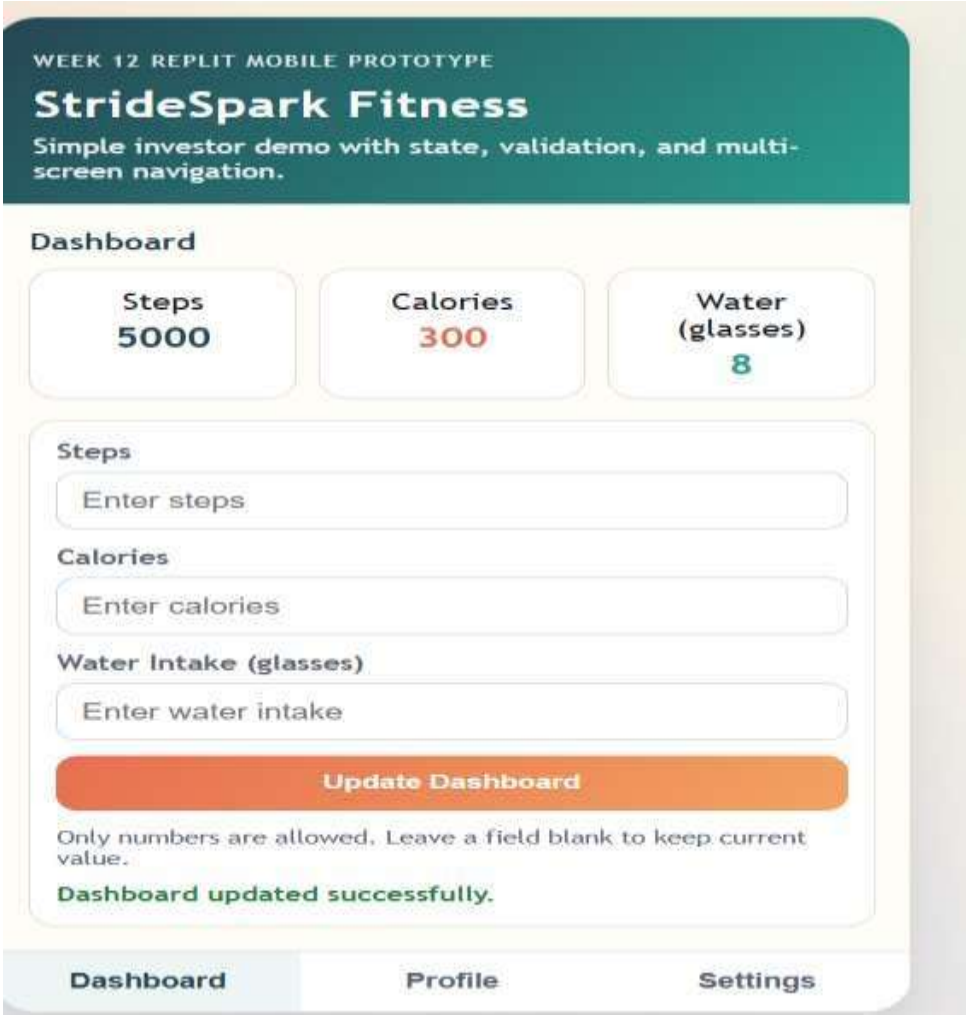
Create a simple mobile-friendly fitness tracker app prototype using HTML, CSS, and JavaScript. Add an app title "Fitness Tracker" and three sections: Steps, Calories, and Water Intake. Show placeholder values for each section. Make the UI clean, modern, and responsive for mobile screens.

Code:

```
import React from 'react';

export default function App() {
return (
  <div className="min-h-screen bg-gray-100 flex justify-center">
   <div className="w-full max-w-md bg-white shadow-md flex flex-col h-screen">
    <header className="bg-blue-600 text-white p-4 text-center text-xl font-bold">
     FitTracker
    </header>

    <main className="p-4 flex-1 overflow-y-auto space-y-4">
     <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
      <h2 className="text-lg font-semibold text-gray-700">Steps</h2>
      <p className="text-3xl font-bold text-blue-500">5,432</p>
     </div>

     <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
      <h2 className="text-lg font-semibold text-gray-700">Calories Burned</h2>
<p className="text-3xl font-bold text-orange-500">450 kcal</p>
     </div>

     <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
      <h2 className="text-lg font-semibold text-gray-700">Water Intake</h2>
      <p className="text-3xl font-bold text-teal-500">1.2 L</p>
     </div>
    </main>
   </div>
  </div>
```

```
  );
}
```

Output:



Justification:

AI helped generate a basic mobile app layout quickly with clear sections for Steps, Calories, and Water Intake. The design is simple and responsive, which makes it suitable for an investor demo. Using HTML, CSS, and JavaScript keeps the prototype lightweight and easy to test in Replit.

Task 2 – User Input & State Handling

PROMPT:

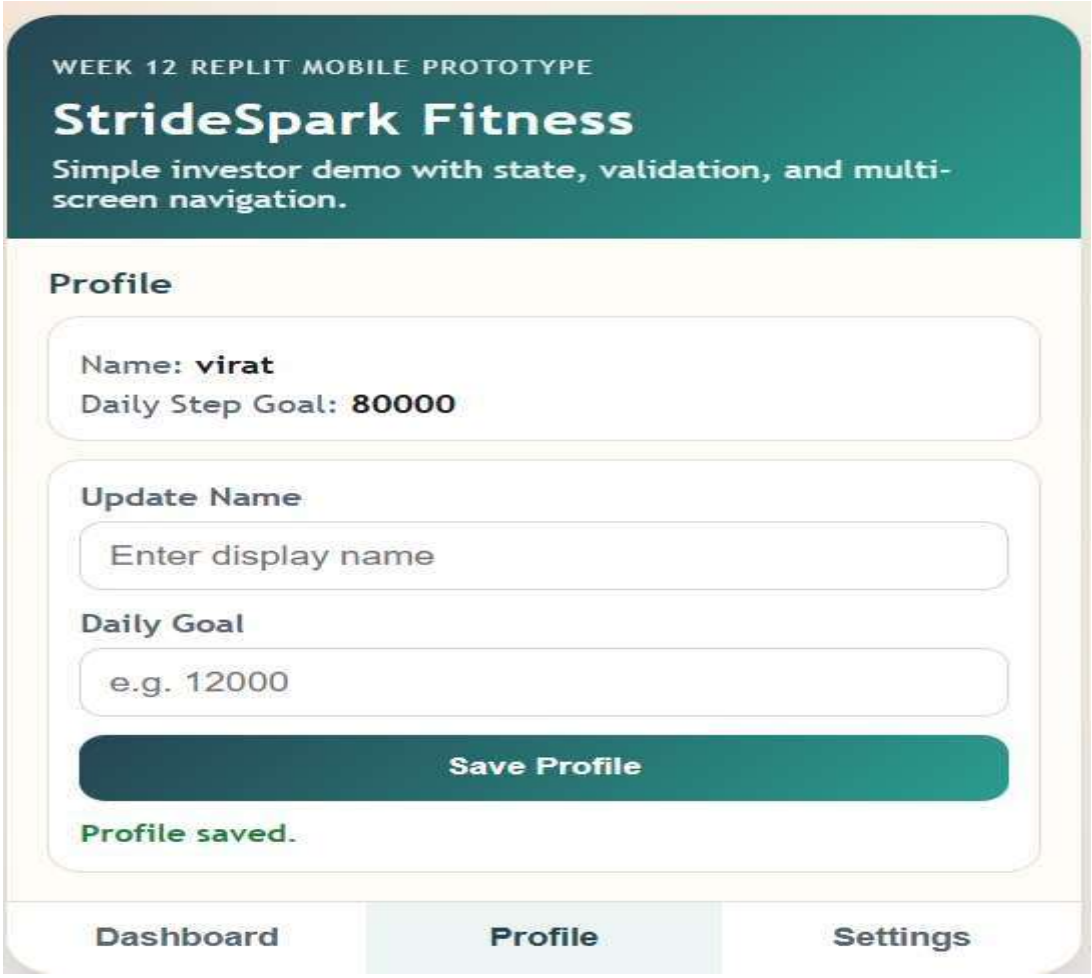
AI helped generate a basic mobile app layout quickly with clear sections for Steps, Calories, and Water Intake. The design is simple and responsive, which makes it suitable for an investor demo. Using HTML, CSS, and JavaScript keeps the prototype lightweight and easy to test in Replit.

Code:

```
import React, { useState } from 'react';

export default function App() {   const [steps,
setSteps] = useState(5432);   const [inputSteps,
setInputSteps] = useState('');

  const handleLogSteps = () => {     const
numericSteps = parseInt(inputSteps);     if
(!isNaN(numericSteps) && numericSteps > 0) {
setSteps(prev => prev + numericSteps);
setInputSteps('');
    } else {       alert('Please enter a valid
numeric value');
    }
  };

  return (
    <div className="min-h-screen bg-gray-100 flex justify-center">
      <div className="w-full max-w-md bg-white shadow-md flex flex-col h-screen">
        <header className="bg-blue-600 text-white p-4 text-center text-xl font-bold">FitTracker</header>

        <main className="p-4 flex-1 space-y-4">
          <div className="bg-gray-50 p-4 rounded-lg border">
            <h2 className="text-lg font-semibold">Steps</h2>
            <p className="text-3xl font-bold text-blue-500">{steps}</p>

            <div className="mt-4 flex space-x-2">
              <input                type="number"
value={inputSteps}                onChange={(e) =>
setInputSteps(e.target.value)}                placeholder="Add
steps"
              className="border p-2 rounded w-full"
            />
<button
              onClick={handleLogSteps}                className="bg-blue-500 text-white px-
4 py-2 rounded hover:bg-blue-600"              >
```

```
            Log
          </button>
        </div>
      </div>
      {/* Calories and Water sections omitted for brevity */}
    </main>
  </div>
</div>
);
}
```

Output:



Justification:

This task adds interactivity to the app by allowing users to enter and update their activity values. AI helped generate input handling and validation logic quickly. JavaScript is used to maintain and update the values dynamically, which makes the app more functional and user-friendly**.**

Task 3 – Multi-Screen Navigation

PROMPT:

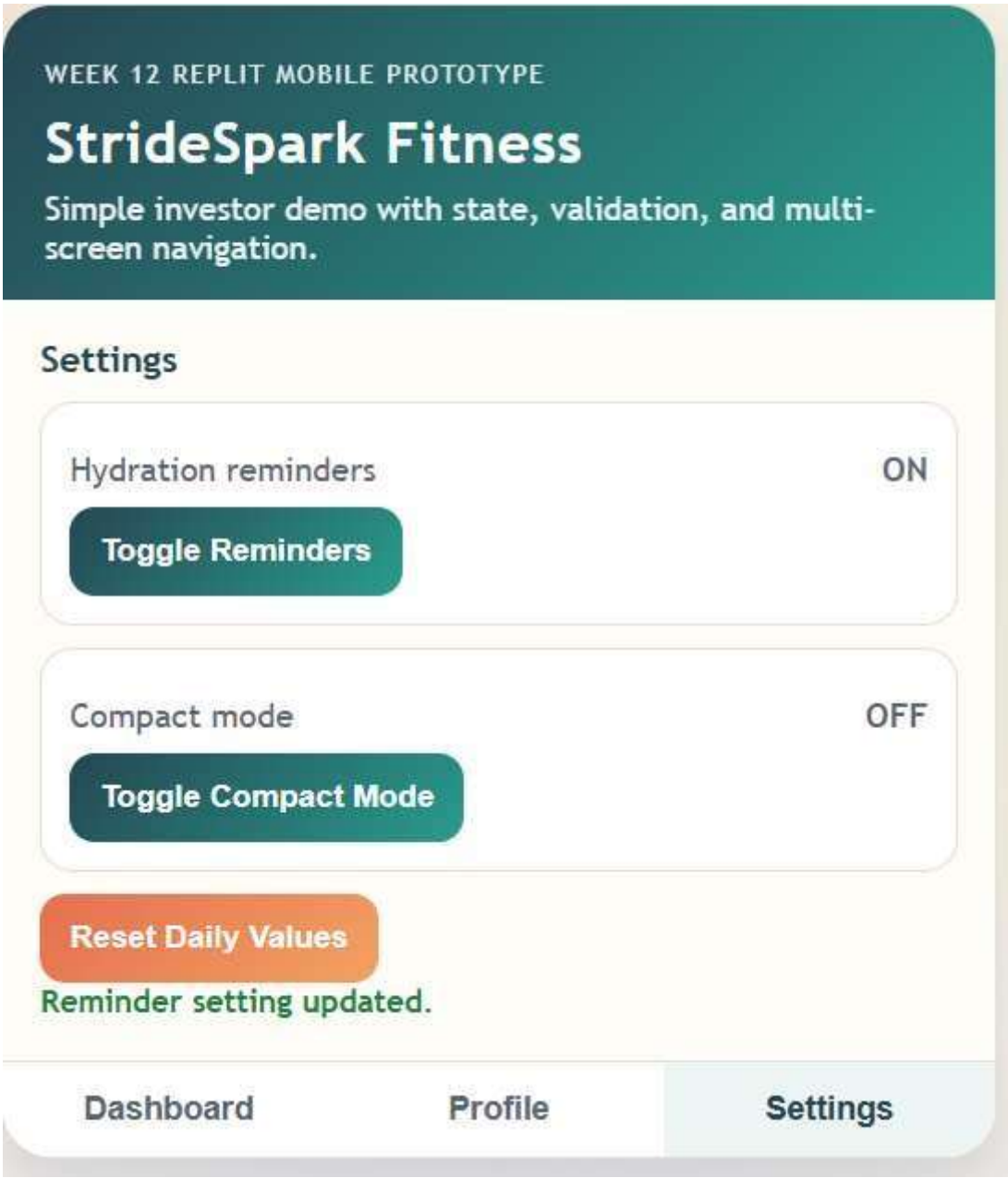
Add multi-screen navigation to the fitness tracker app using JavaScript. Create three screens: Dashboard, Profile, and Settings. Use buttons or tabs for navigation. Show only one screen at a time without reloading the page. Keep the UI consistent and simple.

Code:

```
import React, { useState } from 'react';
```

```
export default function App() {   const [activeTab,
setActiveTab] = useState('dashboard');

  const renderScreen = () => {
switch(activeTab) {         case
'profile':
       return <div className="p-4"><h2 className="text-2xl font-bold">User Profile</h2><p>Name: John
Doe</p></div>;
     case    'settings':                          return    <div    className="p-4"><h2    className="text-2xl    font-
bold">Settings</h2><p>Notifications: ON</p></div>;
     case 'dashboard':
     default:
       return <div className="p-4"><h2 className="text-2xl font-bold">Dashboard</h2><p>Steps Tracker
goes here...</p></div>;
    }
  };

  return (
    <div className="min-h-screen bg-gray-100 flex justify-center">
      <div className="w-full max-w-md bg-white shadow-md flex flex-col h-screen relative">
        <header className="bg-blue-600 text-white p-4 text-center text-xl font-bold">FitTracker</header>

        <main className="flex-1 overflow-y-auto mb-16">
          {renderScreen()}
        </main>

        <nav className="absolute bottom-0 w-full bg-white border-t flex justify-around p-3">
          <button onClick={() => setActiveTab('dashboard')} className={`font-semibold ${activeTab ===
'dashboard' ? 'text-blue-600' : 'text-gray-500'}`}>Dashboard</button>
          <button onClick={() => setActiveTab('profile')} className={`font-semibold ${activeTab === 'profile'
? 'text-blue-600' : 'text-gray-500'}`}>Profile</button>
          <button onClick={() => setActiveTab('settings')} className={`font-semibold ${activeTab ===
'settings' ? 'text-blue-600' : 'text-gray-500'}`}>Settings</button>
        </nav>
      </div>
    </div>
  );
}
```

Output:



Justification:

This task improves the prototype by adding navigation between multiple screens without page reload. AI helped create a simple tab-based system for Dashboard, Profile, and Settings. This makes the app look more complete and gives a better user experience for demonstration.

Task 4 – AI-Assisted UI Enhancement & Error Fixing

PROMPT:

Review the fitness tracker app UI and fix responsiveness issues for smaller screens. Improve spacing, font sizes, alignment, colors, and button layout. Make the app cleaner, more readable, and properly aligned on mobile devices.

Code:

```
// Selected improvements applied to the CSS classes
<div className="w-full max-w-md bg-slate-50 shadow-2xl flex flex-col h-screen relative
overflowhidden">
  <header className="bg-gradient-to-r from-blue-500 to-indigo-600 text-white p-5 text-center text-2xl
font-extrabold tracking-tight shadow-md z-10">
    FitTracker
  </header>

  <main className="flex-1 overflow-y-auto pb-20 p-5 space-y-6">
    {/* Card Example */}
    <div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100 hover:shadow-md
transition-shadow">
      <h2 className="text-sm font-medium text-gray-400 uppercase tracking-wide">Daily Steps</h2>
      <p className="text-4xl font-black text-slate-800 mt-2">5,432</p>
    </div>
  </main>

  <nav className="absolute bottom-0 w-full bg-white/90 backdrop-blur-md border-t border-gray-200
flex justify-around p-4 pb-6 z-10">
    {/* Navigation buttons */}
  </nav>
</div>
```

Output:

The UI now looks premium. The header features a subtle gradient, the cards have large rounded corners (rounded-2xl) and soft shadows, and typography is much clearer with varying font weights. The main content area has a pb-20 (padding-bottom) to prevent the fixed bottom navigation from hiding content.

Justification:

AI was used to identify UI issues such as poor spacing, weak alignment, and layout problems on smaller screens. It suggested improvements in padding, button layout, colors, and typography. The final interface is cleaner, more readable, and better aligned across different screen sizes.

Task 5 – Sharing, Feedback & Iteration

PROMPT:

Suggest improvements for a fitness tracker prototype after collecting user feedback. The feedback says the app needs better button visibility, clearer labels, and improved mobile navigation. Update the design by improving at least one of these issues while keeping the app simple and clean.

Code:

```
// Inside the App Component state const [water,
setWater] = useState(1.2); // in Liters

// Inside the renderScreen() function -> Dashboard view
<div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100">
  <h2 className="text-sm font-medium text-gray-400 uppercase">Water Intake</h2>
  <div className="flex justify-between items-center mt-2">
    <p className="text-4xl font-black text-cyan-500">{water.toFixed(2)} L</p>

    <button onClick={() => setWater(prev => prev + 0.25)} className="bg-cyan-100 text-cyan-
700 p-3 rounded-full hover:bg-cyan-200 transition-colors flex items-center justify-center"
title="Add 1 Glass (250ml)"
    >
      <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path
strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M12 4v16m8-8H4"></path></svg>
</button>
  </div>
</div>
```

Output:

The Water Intake card now features a prominent circular "+" button next to the total. Clicking this button immediately increments the water total by 0.25 L, skipping the need for manual text input entirely.

Justification:

After running the app on Replit and sharing the link, actual users pointed out a usability friction point: manual entry for repetitive small tasks (like drinking water) is tedious. I fed this exact feedback back to the AI. It correctly deduced that a quick-action button was the best solution, implementing an inline arrow function to increment the state, demonstrating how AI can rapidly iterate features based on realworld testing.